ADR 0011 — Backend Interface v2

Status: Proposed (Marv to confirm after review) **Date**: 2026-08-21 **Author**: Marv **Related**: ADR 0001, ADR 0002, ADR 0005 (superseded for v2), ADR 0006, ADR 0010, ADR 0012, docs/BACKEND_FLOW.md, docs/METHODOLOGY.md

---

Context

Phase 1 shipped a v1 backend abstraction (ADR 0005) — the `InferenceBackend` protocol with `plan_task` + `interpret_results`, one concrete `OpenAIBackend`. Phase 2 demands a richer surface:

1. **Local MLX inference** as a second backend for the Qwen 2.5 1.5B + LoRA fine-tune (per ADR 0004 base model choice and RONIN_SYNC B-tier decisions)
2. **Retry + resilience** — Ronin's eval fires many requests in parallel; transient 429s and network failures need retry logic that doesn't invite the user to reason about them
3. **Warmup** — MLX cold-start is 5-10s; server must not accept requests before the model is ready
4. **Registry-driven metadata** — model facts (context window, cost model, adapter paths) live in `models/models.json` per ADR 0012; backend must read them
5. **Failure surfacing** — backend errors must map cleanly to `AgentResponse` so scorer can bucket them
6. **Uniformity for research fairness** — every backend must implement identical semantics; agent code must have zero backend-conditional logic (per METHODOLOGY §7)

ADR 0005's minimal-surface protocol works for the OpenAI-only Phase 1 case but doesn't accommodate the above. This ADR is the v2 iteration.

Constraint set:

- FastAPI async is the primary caller (ADR 0002 §implementation)
- Author-split discipline (METHODOLOGY §3) — Marv owns backends, agent, adapter; Ronin doesn't touch this code
- Extended timeline (Nov-Dec 2026 target) — quality over calendar
- Backend classes MUST be per-transport (OpenAI API, MLX local, Anthropic API) not per-model — one class handles all models within a transport

---

Decision

1. Backend hierarchy and factory pattern

`InferenceBackend` remains an ABC in `src/baymax/service/inference.py`. Three concrete classes total, ever:

- `OpenAIBackend` — HTTPS to OpenAI API. Handles all OpenAI-served models via `model_id`.
- `MLXBackend` — local MLX inference. Handles all locally-served Qwen variants + adapters via `model_id`.

- `AnthropicBackend` — deferred (v3+). Not implemented in v1.

Backend selection at app startup uses a factory dict pattern:

```
BACKENDS: dict[str, type[InferenceBackend]] = {
    "openai": OpenAIBackend,
    "mlx": MLXBackend,
}
```

Selection via env var `BAYMAX_BACKEND=openai|mlx`. FastAPI lifespan constructs one backend instance for the process lifetime, passes it to the agent as a constructor dependency (matches ADR 0005 §Decision).

Adding a fourth backend later = one line in `BACKENDS` + one new class file.

2. Method signatures

Base class ABC:

```
class InferenceBackend(ABC):
    model_id: str
    max_context: int          # from registry per §7
    cost_model: str           # "per_token" | "amortized"
    provider: str             # "openai" | "mlx" | ...

    def __init__(self, model_id: str) -> None:
        # Fetches own required env vars (OPENAI_API_KEY, MLX_MODEL_PATH, etc.)
        # Loads registry entry per §7
        # Raises loud on missing env or missing registry entry

    async def plan_task(self, task: TaskFile) -> PlanResult: ...
    async def interpret_results(self, task: TaskFile, results: list[ToolCallResult]) ->
Interpretation: ...
    async def warmup(self) -> None: ...
    async def _call_llm(self, prompt: str, tools: list[Tool]) -> RawLLMResponse: ...  # base impl
with retry
```

Return types are wrapped:

```
@dataclass
class PlanResult:
    steps: list[ToolCallStep]
    tokens_used: int
    cost_usd: float
    warnings: list[str] = field(default_factory=list)

@dataclass
class Interpretation:
    text: str
```

```
    tokens_used: int
    cost_usd: float
```

Naming keeps `plan_task` + `interpret_results` (ADR 0001 Plan-and-Execute pattern). All methods async; sync work (e.g., MLX inference) wrapped via `run_in_executor` inside the backend.

`_call_llm` is a shared base-class helper — centralizes retry logic (§5) and telemetry emission so subclasses don't reimplement.

Streaming intentionally omitted for v1 (deferred to v2 per DEFERRED.md). When added, a new `plan_task_stream()` method rather than a signature break to `plan_task()`.

3. Construction and configuration

- `model_id`: constructor param, passed by FastAPI lifespan (populated from `BAYMAX_MODEL_ID` env). Backend is a pure function of its inputs; tests pass literal strings.
- **Backend selection**: env var `BAYMAX_BACKEND`.
- **Secrets**: each backend fetches its own required env vars in `__init__` and raises loud on missing:
  - `OpenAIBackend`: `OPENAI_API_KEY`
  - `MLXBackend`: `MLX_MODEL_PATH` (or reads from registry entry per §7)
  - `AnthropicBackend` (v3+): `ANTHROPIC_API_KEY`
- **Configuration validation timing**: lifespan startup. FastAPI `lifespan()` constructs the backend, warmup runs, `/health` becomes ready. Anything wrong = process exits before requests are accepted.

Env-var summary:

Env var	Required by	Purpose
`BAYMAX_BACKEND`	Factory	Selects backend class (`openai` or `mlx`)
`BAYMAX_MODEL_ID`	All backends	Model to serve within the selected backend
`BAYMAX_RETRY_MAX_ATTEMPTS`	Base class retry	Defaults to 3; sweep target per §5
`OPENAI_API_KEY`	OpenAIBackend	API auth
`MLX_MODEL_PATH`	MLXBackend	Weights location (or from registry)

4. Warmup semantics

- **When**: during FastAPI lifespan startup, after backend construction, before `lifespan()` yields
- **What**: execute a dedicated warmup scenario from `src/baymax/service/warmup/warmup_scenario.json` (uses the standard scenario schema; forces at least one tool-call generation to warm MLX's structured-output compute graphs)
- **Gate**: `/health` returns `503 {"ready": false, "reason": "warming_up"}` until warmup completes; then `200 {"ready": true, "backend": <str>, "model_id": <str>}`
- **Timeout**: 120s hard cap. Beyond that = deterministic failure (bad weights, wrong path, incompatible format). Raises `WarmupTimeout` → uvicorn exits.

- **Failure handling**: fail-loud. NO retry on warmup errors. All warmup failure modes (missing weights, corrupted checkpoint, format incompatibility, base-model mismatch, MLX version mismatch, insufficient RAM, tokenizer mismatch, Metal unavailable, malformed warmup JSON) are deterministic — retry masks real problems. Errors raise `WarmupFailure(stage, cause, hint)` for diagnostic clarity.
- **Telemetry tag**: warmup call emits with `is_warmup: true` so Ronin's eval runner can filter it out of eval score aggregations.

5. Retry policy

`_call_llm` implements retry with the following semantics:

- **Max attempts (N)**: **hardcoded to 3.** Simplified from earlier "configurable + swept in eval" decision (2026-08-24 revision) — publication-review-oriented sweep infrastructure was overkill for an artifact-first project. If retry sensitivity ever becomes a real concern, promote to configurable then.
- **Backoff formula**: `delay = 1s × (2 ^ attempt)` → 1s, 2s, 4s. Plus ±25% random jitter to prevent synchronized retries from parallel eval batches.
- **Retriable errors** (API backends only):
  - HTTP 408 (Request Timeout)
  - HTTP 429 (Rate Limit)
  - HTTP 500-504 (server errors)
  - Connection errors (`ConnectionResetError`, `ConnectTimeoutError`, DNS failures)
- **Non-retriable errors** (fail immediately):
  - HTTP 400 (Bad Request — includes OpenAI content-policy violations)
  - HTTP 401 (Unauthorized)
  - HTTP 403 (Forbidden)
  - HTTP 404 (Not Found)
  - HTTP 422 (Unprocessable Entity)
- **MLX errors**: NO retry, ever. All MLX errors are deterministic (memory pressure, model config, config drift) — retry doesn't self-heal on the seconds of a retry window. Same principle as warmup fail-loud.
- **Telemetry**: BOTH per-attempt events AND final aggregate emitted. Per-attempt for debugging (`event: "backend_attempt"`, `event: "backend_retry"`); aggregate for accounting (`event: "backend_call_complete"`, `total_attempts: N`, `final_status`).

6. Error surfacing

- **Exception type**: `BackendError` in `src/baymax/core/exceptions.py`. Wraps underlying exception always (even non-retry failures). Has `kind`, `underlying`, `attempts_history`, `backend` fields.
- `kind` **values** (extensible enum-of-strings, not exhaustive):
  - `"retry_exhausted"` — retriable error hit `max_attempts` without success
  - `"non_retriable"` — first-attempt error was classified as non-retriable (HTTP 400/401/403/404/422, or any MLX error)
  - `"unreachable"` — defensive guard for the retry loop exiting without return/raise (should never fire in practice)

- **Agent-side internal state**: on `BackendError`, agent sets `TaskFile.completion_status = FAILED` and `TaskFile.error_type = ErrorType.BACKEND_ERROR` (new enum value added to `contracts.py`). Full underlying + attempt history retained in the action log for forensic value. `AgentResponse` boundary shape is NOT modified — remains `{tool_calls, message}` per its scorer contract.
- **HTTP status (revised 2026-08-26)**: `/invoke` returns **HTTP 500** with a JSON error body when `BackendError` propagates. Rationale: backend failure is infrastructure-layer, not an agent decision. Clean separation:
  - `200 + AgentResponse` = agent made a decision (execute, refuse, clarify)
  - `5xx + error body` = infrastructure failed (backend down, retries exhausted, warmup incomplete)
  - Aligns with HTTP semantics; keeps `AgentResponse` boundary contract intact for Ronin's scorer.
- **500 body shape**: `{"error": "backend_error", "kind": <retry_exhausted|non_retriable>, "backend": <provider>, "message": <str>, "attempts": <int>}`. No stack traces, no `underlying` repr in the body — those go to the trace file only, per redaction discipline (ADR 0010).
- **FastAPI wiring**: `app.exception_handler(BackendError)` registered in `api.py`'s lifespan. Handler builds the 500 body from `exc.kind` / `exc.backend` / `len(exc.attempts_history)`. Agent code itself does NOT catch `BackendError` — lets it bubble up.
- **Retry history in trace**: yes, full per-attempt events emitted per §5 propagate into the trace stream.
- **Ronin coordination note**: eval runner needs a try/except around the httpx call and a "backend_error" bucket in categorization. Small change on his side; flag in next sync.

7. Registry coupling

Backend reads the required registry fields at construction time (see ADR 0012 §2):

- `id` (model identifier)
- `base` (base model)
- `adapter_path` (nullable for non-adapter models)
- `trained_at`
- `training_config_hash` (reproducibility trail; may be named `config_hash` per Ronin's naming — see ADR 0012 sync note)

Backend does NOT hardcode any per-model facts. Adding a new fine-tuned model = registry entry update, not a backend code change.

Runtime facts NOT stored in registry (per ADR 0012 §2 trimmed optional list):

- `max_context` — read from HF config of the base model at `__init__` time
- `cost_model` — hardcoded per backend class (`OpenAIBackend.cost_model = "per_token"`, `MLXBackend.cost_model = "amortized"`)
- `tokenizer_path` — inferred from the base model directory / HF config

Consequence: registry lookup happens ONCE at `__init__` time; supplementary facts loaded from the base model's own config in the same phase. Backend caches everything as instance attributes for the process lifetime. No per-request registry reads.

8. Testing

- **Unit tests**: use `FakeBackend` in `tests/fakes/backend.py`. Canned responses via constructor args. Registered in the `BACKENDS` dict under `"fake"` for tests that spin up the full FastAPI app.
- **Retry tests**: patch `asyncio.sleep` so backoff delays don't slow the unit test suite.
- **Warmup gate tests**: `FakeBackend(model_id="fake", warmup_delay_s=0.5)` — injectable delay makes the 503→200 transition observable.
- **Integration tests**:
  - OpenAI: opt-in via `pytest.mark.integration` and `[test-integration]` PR tag. Requires `OPENAI_API_KEY`. Auto-retry once on transient flake, then fail loud.
  - MLX: Marv-local only. `pytest.mark.mlx_local`, skipped unless `HAVE_MLX=1`. Not in CI (Ahmed doesn't have Mac hardware; MLX is Marv-owned per OWNERSHIP.md).
- **No nightly scheduled runs**. Ahmed's eval runs during Phase 2 naturally catch drift. Project ends Dec 2026 — nightly overkill.

Reasoning

[REASONING PROMPTS — expand each bullet into 1-2 sentences in your own voice. Documentation depth: brief, honest, readable in 6 months (artifact-first project, per 2026-08-24 scope revision).]

- **WHY template-per-TRANSPORT, not per-model** (OpenAIBackend handles all OpenAI models, MLXBackend handles all Qwen variants): OpenAIBackend already handles gpt-4o-mini + gpt-4o + gpt-4-turbo as one class. Same logic for MLX. 3-4 classes total ever = manageable. Class-per-model would explode into dozens of near-duplicates that must all be kept in sync — SOLID violation of the wrong kind.
- **WHY enforce interface uniformity rather than advertise** `.capabilities()`: research fairness. If the agent branches based on backend capabilities, the eval-score delta between backends is contaminated by agent-code-path differences — you can't isolate "the model" as the variable. Every backend absorbs its own capability gaps internally; the agent code has ZERO backend-conditional logic. Descriptive metadata (`max_context`, `cost_model`) IS exposed for telemetry, just not consumed by agent logic.
- **WHY** `_call_llm` **on the base class rather than duplicated per backend**: retry logic is cross-cutting; centralizing it in one place means retry policy can't drift between backends. Every retry rule, every backoff calculation, every telemetry emission happens exactly once. Duplicating retry per backend guarantees they'll diverge subtly and someone will chase a bug through three implementations.
- **WHY retry N = 3 hardcoded** (2026-08-24 revision, replaces earlier configurable+swept decision): the retry-sensitivity sweep was publication-review flavor — turning "is retry a variable?" into a formal empirical measurement. For an artifact-first project, N=3 matches SDK conventions (OpenAI, Anthropic SDKs) and is enough resilience for eval batch runs without overengineering. If retry sensitivity ever becomes a real concern for the artifact story, promote to configurable then.
- **WHY hardcode base delay + factor + jitter**: same principle as N=3 — SDK-convention values (1s base, 2x factor, ±25% jitter) work fine. Configurability adds knobs no one is going to turn.
- **WHY MLX gets NO retry, ever** (including OOM and inference timeout): MLX errors are deterministic. Metal OOM's freeing event isn't synchronized to your 7-second retry window; MLX inference timeout on the same prompt and same infrastructure will time out again. Retry masks systemic issues (memory pressure, config drift) that need to fail loud so the human fixes root cause. Same principle as warmup fail-loud.

- **WHY the retry policy asymmetry is honest, not a bug**: API backends have network-transient errors that self-heal on external clocks (rate windows reset, servers recover). MLX has local-and-deterministic errors that don't self-heal on retry timescales. Modeling that asymmetry accurately IS the honest choice; making them symmetric would either mask real API problems (N=1 on OpenAI inflates failure rate on 429s) or waste time on MLX (retry adds nothing). The asymmetry is reported honestly per METHODOLOGY §6 statistical requirements.
- **WHY custom** `BackendError` **always** (wraps even non-retry errors): agent has ONE exception type to handle. Cross-backend debugging is easier when errors have consistent shape. Original exception preserved as `underlying` — no forensic loss. Different exception types per backend means agent code needs backend-conditional catches, which violates §7 uniformity.
- **WHY new** `ErrorType.BACKEND_ERROR` **enum value** rather than reusing `INTERNAL_ERROR`: backend failures are a distinct failure mode from agent-logic failures (validator errors, missing arguments, ambiguous requests). Ronin's scorer needs to bucket them separately. Reusing an existing enum value = losing information the eval needs.
- **WHY warmup uses a dedicated JSON file** (rather than hardcoded string): reuses your existing scenario schema — no new format to maintain. Editable without a code change. Doesn't point at real eval scenarios (avoids confusion of "why is this eval scenario running at startup?"). Consistent with the "everything is data" posture.
- **WHY warmup MUST exercise tool-call generation**: MLX may compile separate compute graphs for structured-output paths (tool-call formatting) vs plain text completion. A "hello world" warmup prompt only warms one path; first real request that requires tool-calling still eats the cold-graph latency. Warmup scenario is designed to force at least one tool call.
- **WHY warmup blocks** `/health` **and warmup failure exits uvicorn**: server that reports healthy while unable to serve requests is worse than server that reports unhealthy. Load balancers, eval harnesses, k8s probes all correctly interpret 503 as "not ready" and wait or route elsewhere. Startup failure = uvicorn exit with clear error = deploy correctly fails; alternative (server accepts requests then errors on first real work) makes drift look like intermittent flakiness.
- **WHY registry-driven metadata rather than backend-internal**: registry is single source of truth per METHODOLOGY §8. Duplication of facts in backend code = drift between backend and registry. Adding a new fine-tune = registry entry + weights drop = backend picks it up. Backend-internal knowledge means every new model is a code change, which slows Ronin's training iteration.
- **WHY lifespan-startup validation over import-time or first-request**: import-time = testing pain (unit tests unrelated to backend selection need env vars set to import the module). First-request = user-visible failure (server reports healthy but errors on first work). Lifespan is the layer where "starting the server" concerns belong — fails before requests accepted, doesn't break test imports.
- **WHY per-attempt AND aggregate telemetry (both)**: per-attempt is real signal for eval analysis and debugging ("45% of scenarios needed 2+ attempts on OpenAI" changes analysis). Aggregate is for eval-scoring accounting ("how many completed successfully overall"). Consumer filters at query time — Marv wants debug view, Ronin wants score view; both from the same emitted stream.
- **WHY FakeBackend is a dedicated class rather than mocking via** `unittest.mock`: explicit fake makes test intent readable. Multi-step call sequences (return this then that) are cleaner with a stateful fake than with mock side-effects. The factory dict pattern makes wiring trivial.
- **WHY on-demand integration tests, not nightly**: project ends Dec 2026. Silent drift probability over 4 months is very low. Ahmed's regular eval runs already function as integration tests. Nightly scheduled runs = cost + noise for capability the project doesn't need.

- **WHY MLX integration is Marv-local, not in CI**: Ahmed doesn't have Mac hardware; GitHub Actions macOS runners cost ~10x Linux minutes plus need weights downloaded per run. MLX ownership is Marv's per OWNERSHIP.md; asymmetric test coverage is acceptable for a two-person project with clean boundaries.

Alternatives considered

Alternative A — One backend class per model (rejected)

Rather than `MLXBackend(model_id="qwen-1.5b-baymax-v1")`, have `Qwen15BBaymaxV1Backend`, `Qwen15BBaymaxV2Backend`, `Qwen7BBackend` as separate classes.

Why rejected: massive code duplication for backends that share the same transport (MLX process management, tokenization, prompt formatting). Adding a fifth model = new class + new tests + risk of divergence. Backends should be per-transport; model is just a config param.

Alternative B — Advertise capabilities via `.capabilities()` (rejected)

Each backend exposes `capabilities() -> dict` with `{streaming, native_tool_calling, max_context, cost_model}`; agent branches on these.

Why rejected: contaminates the research comparison. If the agent branches based on backend capabilities, eval-score deltas between backends are attributable to agent-code-path differences, not model quality. Research position (METHODOLOGY §1) demands agent code has zero backend-conditional logic. Descriptive metadata is still EXPOSED — just for telemetry/debug, not consumed by agent logic.

Alternative C — Sync-only interface (rejected)

Backends are sync; wrap in `run_in_threadpool` at the agent layer.

Why rejected: FastAPI is async. Every sync backend call would block the event loop or require thread-pool overhead at the agent layer for every backend. Making backends async and letting them wrap sync work internally (via `run_in_executor` for MLX) keeps the calling convention uniform and the event loop unblocked.

Alternative D — Backend-internal model metadata (rejected)

Each backend has hardcoded knowledge per model:

```
MLXBackend.MODEL_METADATA = {
    "qwen-1.5b-baymax-v1": {"max_context": 32768, "adapter_path": "..."},
    "qwen-1.5b-baymax-v2": {"max_context": 32768, "adapter_path": "..."},
}
```

Why rejected: adding a new fine-tune requires a backend code change, which slows Ronin's training iteration. Registry drift (backend's hardcoded facts vs `models/models.json` reality) becomes a real bug source. Single source of truth (registry) prevents drift by construction.

Alternative E — Non-configurable retry N (rejected)

Fix N = 3 as a constant.

Why rejected: leaves the "does retry meaningfully affect failure rate?" question unanswered. Making N configurable + sweeping in eval turns opinion into measurement. If the sweep shows N doesn't matter, we can hardcode later with data behind the decision.

Alternative F — Retry on MLX errors too (rejected)

Symmetric retry policy: both OpenAI and MLX retry on all failures.

Why rejected: MLX errors are deterministic. Metal OOM's freeing event isn't synchronized to retry windows. Retry masks systemic issues (memory pressure, config drift) that need to fail loud. Modeling infrastructure reality accurately (API = network-transient, MLX = local-deterministic) is more honest than forced symmetry.

Consequences

Positive

- Adding a new backend transport (Anthropic, other API providers) = one line in `BACKENDS` dict + one new class. Trivial extensibility.
- Adding a new fine-tuned MLX model = registry entry + weights drop. Zero code change.
- Retry logic centralized in base class = one place to reason about resilience, no drift.
- Failure surfaces are uniform (all `BackendError`); agent code doesn't need per-backend catches.
- Warmup gate on `/health` composes cleanly with load balancers and orchestration.
- Retry sensitivity swept in eval = data-driven answer to a research-fairness question.
- Testing surface is well-defined; FakeBackend enables full unit coverage of the agent without touching real APIs.
- Registry-driven metadata makes model provenance auditable.

Negative

- Two more env vars to configure (`BAYMAX_BACKEND`, `BAYMAX_MODEL_ID`) — small but real setup cost per deployment.
- Base-class `_call_llm` couples retry policy to the base class; changing retry semantics = base class change (though this is arguably positive — hard to accidentally diverge).
- Warmup adds 5-10s to server startup for MLX (unavoidable cost of local inference).
- Retry asymmetry (API retries, MLX doesn't) needs to be documented alongside every eval comparison so reviewers understand.
- Registry becomes a load-bearing dependency; corrupt or missing `models.json` at startup = process exit (though this is arguably positive — surfaces the problem loudly).
- Integration tests are asymmetric (OpenAI in CI on-demand; MLX Marv-local only). Acceptable given ownership boundaries but not scale-friendly.

Neutral

- Backend selection is per-process, not per-request. A single process serves one backend for its lifetime. If we ever need multi-backend routing (e.g., small-model triage → large-model fallback), that would require a new ADR.

- Streaming remains deferred to v2. When added, extends the interface via new methods rather than breaking existing ones.

Open questions / follow-ups

- **Backend capability declaration expansion**: if a fourth backend surfaces genuine capability divergence that can't be absorbed internally, may need to revisit the enforcement-vs-advertisement decision. Not v1 concern.
- **Backend selection at runtime**: currently per-process via env var. If Ronin's eval ever wants to compare A/B backends in a single run (rather than separate processes), the design would need a per-request `backend` param. Deferred until concrete requirement.
- **Retry policy per-error-class tuning**: currently uniform backoff for all retriable errors. If certain errors benefit from different backoff (e.g., 429 with `Retry-After` header), specialize later.
- **Warmup scenario evolution**: single hardcoded warmup JSON. If MLX or fine-tuned models need model-specific warmup prompts, extend to per-model warmup scenarios.
- **MLX conversion pipeline**: model registry entries with `mlx_compatible: false` need a conversion step before MLX can load them. Who owns conversion (Marv side, Ronin side, or automated) — resolve in ADR 0012 or a new ADR.
- **Anthropic backend**: deferred to v3+; when it lands, verify the enforcement principle still holds (interface uniformity absorbable in `AnthropicBackend` internals).

References

- ADR 0001 — Plan-and-Execute pattern this interface serves
- ADR 0002 — boundary contract for AgentResponse
- ADR 0005 — v1 predecessor (superseded for v2 by this ADR)
- ADR 0006 — telemetry format used by `_call_llm` emission
- ADR 0010 — telemetry infrastructure
- ADR 0012 — registry this backend reads from
- docs/BACKEND_FLOW.md — visual reference diagrams
- docs/METHODOLOGY.md — §7 uniformity, §8 registry integration
- RONIN_SYNC.txt B-tier — locked Phase 2 technical decisions
- Design walkthrough transcript — chat session 2026-07-27 through 2026-08-21 with Claude

---

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.